



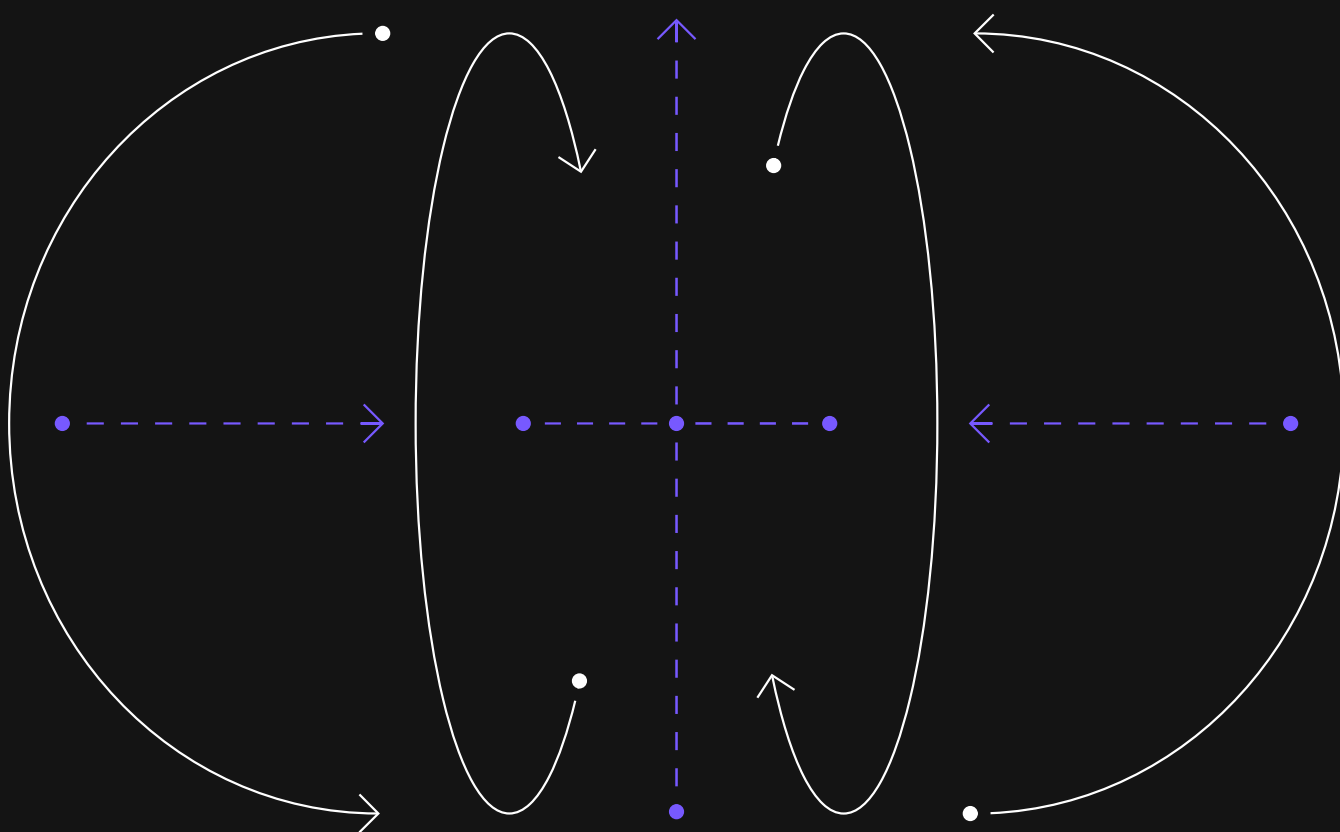
Машинное обучение

Лонгрид

Методы оптимизации

03

Автоматическое дифференцирование



Автоматическое дифференцирование

Контекст

Почти все методы оптимизации первого и второго порядка работают не с самой функцией L , а с её локальной моделью: градиентный спуск идёт вдоль $-L'$, квазиньютоновские методы приближают кривизну, шаг в методе Ньютона получается из минимизации квадратичной аппроксимации. Значит, прежде чем обсуждать сами методы, нужно уметь дёшево и устойчиво считать производные/градиенты. При $d \sim 10^6-10^{12}$ параметрах аналитические формулы и конечные разности перестают быть рабочим инструментом.

Применение

Автоматическое дифференцирование (AD) — это то, что стоит за `torch.autograd`, `jax.grad` и любым современным обучением нейросетей. Понимая forward/reverse-подходы к вычислению градиентов/якобианов, JVP/VJP и сложность по памяти каждого из подходов, ты заранее знаешь, какой режим выбрать для полного якобиана, почему градиент скалярного лосса почти бесплатен по времени и откуда берётся OOM на глубоких моделях.

Задача лонгрида

Свяжем правило вычисления производной сложной функции с вычислительным графом, разберём reverse-AD-подход на примере вычисления attention в трансформерной архитектуре, сравним вычислительные сложности подходов и перейдём к главному практическому узкому месту reverse-подхода — росту потребления памяти для хранения промежуточных значений функций.

Краткий план

- Зачем AD и чем плохи альтернативы
- Вычислительный граф, JVP и VJP
- Reverse на attention
- Выбор способа обхода графа для вычисления якобиана, вычисление hessian-vector product (HVP)
- Память и чек-пойнтинг

1. ТРИ СПОСОБА ВЗЯТЬ ПРОИЗВОДНУЮ

Хотим решать $L(\mathbf{w}) \rightarrow \min_{\mathbf{w} \in \mathbb{R}^d}$. Методы нулевого порядка видят только значения L , первого — градиент L' , второго — гессиан или произведения $\mathbf{H}\mathbf{v}$. Вопрос лонгрида: как получить L' (и производные более высокого порядка), если L задана программой?



Есть три принципиально разных пути. **Численно** — конечные разности по значениям L . **Символьно** — правила дифференцирования на аналитической формуле. **Автоматически** — те же правила, но применённые к *вычислительному графу* программы L , а не к замкнутой формуле. AD даёт точность машинной арифметики и (reverse-подход) стоимость градиента порядка стоимости самого L .

	Численно	Символьно	AD
Идея	конечные разности	правила на формулах	правила на графе
Точность	$\sim \varepsilon_{\text{mach}}$	точная	точная (в FP)
Стоимость L'	$\mathcal{O}(dT)$	взрыв выражений	$\mathcal{O}(T)$ (reverse)
Пример	<code>scipy.approx_fprime</code>	SymPy	torch, jax

Здесь $T = \text{OPS}\{L\}$ — стоимость одного вычисления $L(\mathbf{w})$.

Конечные разности

Правая разность по координате:

$$\frac{\partial L}{\partial w_k}(\mathbf{w}) \approx \frac{L(\mathbf{w} + \varepsilon \mathbf{e}_k) - L(\mathbf{w})}{\varepsilon}.$$

Центральная схема точнее:

$$\frac{\partial L}{\partial w_k}(\mathbf{w}) \approx \frac{L(\mathbf{w} + \varepsilon \mathbf{e}_k) - L(\mathbf{w} - \varepsilon \mathbf{e}_k)}{2\varepsilon}.$$

ЗАМЕЧАНИЕ

Цена полного градиента — $\sim dT$ (или $2dT$ для центральной схемы). Выбор ε : слишком большое — ошибка аппроксимации, слишком маленькое — шум от округления. На графике относительной ошибки от ε обычно получается U-образная кривая. Для методов нулевого порядка зависимость от d в оценках сходимости тоже не убраться: без настоящего градиента большие задачи решать дорого.

Символьное дифференцирование

Система правил на выражении $L(\mathbf{w})$ строит формулу для L' . Так устроены компьютерная алгебра и CAS-системы: правило цепи, табличные производные, упрощение выражения¹. Для коротких формул это удобно; для программ с циклами, ветвлениями и тысячами операций выражение производной раздувается (*intermediate expression swell*²), и результат всё равно приходится снова превращать в код. AD, наоборот, применяет «правила на графе вычислений», не собирая гигантскую замкнутую формулу.

На практике символьные производные дают, среди прочего, SymPy³, Mathematica и Maple; в численных пайплайнах ML их почти не используют именно из-за взрыва выражений и слабой стыковки с императивным кодом.

ТЕОРЕМА 1

Если вычисление скалярного $L(\mathbf{w})$ стоит T , то существует алгоритм (обратный режим AD), который считает полный градиент $L'(\mathbf{w})$ за $\mathcal{O}(T)$ операций⁴. Константа небольшая: на практике полный градиент сравнивают с несколькими вызовами L .

¹ Обзор трёх парадигм (численные разности, символьное дифференцирование, AD) и их места в ML: Baydin A. G., Pearlmutter B. A., Radul A. A., Siskind J. M. *Automatic differentiation in machine learning: a survey*. JMLR, 18(153):1–43, 2018. jmlr.org/papers/v18/17-468.html.

² Классическое обсуждение раздувания промежуточных выражений в компьютерной алгебре: Moses J. *Algebraic simplification: a guide for the perplexed*. Communications of the ACM, 14(8):527–537, 1971; см. также Geddes K. O., Czapor S. R., Labahn G. *Algorithms for Computer Algebra*. Springer, 1992.

³ Meurer A. et al. *SymPy: symbolic computing in Python*. PeerJ Computer Science, 3:e103, 2017. [doi:10.7717/peerj-cs.103](https://doi.org/10.7717/peerj-cs.103).

⁴ Linnainmaa S. *Taylor expansion of the accumulated rounding error*. BIT, 16(2):146–160, 1976.

ПРИМЕР 1

Именно эта теорема объясняет, почему обучают модели с миллиардами параметров: один шаг SGD стоит порядка одного-нескольких forward, а не d forwards, как у конечных разностей.

2. ВЫЧИСЛИТЕЛЬНЫЙ ГРАФ, JVP И VJP

Программа L — композиция элементарных операций $(+, \times, \sin, \exp, \log, \text{matmul}, \dots)$. У каждой операции известна производная. Вершины графа — промежуточные значения v_i , рёбра — зависимости «вход $\rightarrow$ выход». AD применяет правило сложной функции к этому графу.

Напомним цепочку. Пусть $h(\mathbf{x}) = f(g(\mathbf{x}))$, $g: \mathbb{R}^n \rightarrow \mathbb{R}^m$, $f: \mathbb{R}^m \rightarrow \mathbb{R}$, $\mathbf{y} = g(\mathbf{x})$. Тогда

$$h'(\mathbf{x}) = \mathbf{J}_g(\mathbf{x})^\top f'(g(\mathbf{x})),$$

где $(\mathbf{J}_g)_{ij} = \partial g_i / \partial x_j$.

Для K слоёв: $\mathbf{v}_0 = \mathbf{x} \in \mathbb{R}^{n_0}$, $\mathbf{v}_i = f_i(\mathbf{v}_{i-1})$, $f_i: \mathbb{R}^{n_{i-1}} \rightarrow \mathbb{R}^{n_i}$, итог $\mathbf{v}_K \in \mathbb{R}^{n_K}$. Полный якобиан

$$\mathbf{J} = \mathbf{J}_K \mathbf{J}_{K-1} \cdots \mathbf{J}_1, \quad \mathbf{J}_i = \frac{\partial \mathbf{v}_i}{\partial \mathbf{v}_{i-1}}.$$

Считать $\mathbf{J}$ целиком дорого; считают по столбцам или по строкам.

ОПРЕДЕЛЕНИЕ 1

Для отображения f_i в точке $\mathbf{v}_{i-1}$:

$$\text{JVP}(f_i, \mathbf{v}_{i-1}, \mathbf{u}) = \mathbf{J}_i \mathbf{u} \quad (\text{Jacobian-vector product}),$$

$$\text{VJP}(f_i, \mathbf{v}_{i-1}, \mathbf{w}) = \mathbf{J}_i^\top \mathbf{w} \quad (\text{vector-Jacobian product}).$$

Прямой режим (forward) накапливает столбцы: $\mathbf{J} \mathbf{e}_j$ цепочкой JVP. **Обратный режим** (reverse/backward) накапливает строки: $\mathbf{e}_i^\top \mathbf{J}$ цепочкой VJP.



Ключевой инженерный факт: для большинства элементарных операций JVP и VJP реализуются без явной сборки $\mathbf{J}_i$. Например, для $\mathbf{v} = \mathbf{A} \mathbf{u}$ имеем $\text{JVP} = \mathbf{A} \dot{\mathbf{u}}$, $\text{VJP} = \mathbf{A}^\top \dot{\mathbf{v}}$. Именно поэтому AD дешёв: на каждом ребре платим за пару матрично-векторных действий того же порядка, что и сама операция.

3. ПРИМЕР: SCALED DOT-PRODUCT ATTENTION

Типичный блок трансформера реализует механизм **внимания**¹. При длине последовательности n и размерности d заданы матрицы $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{n \times d}$. Выход $\mathbf{O}$ вычисляется как

$$\mathbf{O} = \text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathbf{P} \mathbf{V}, \quad \mathbf{P} = \text{softmax}\left(\frac{\mathbf{Q} \mathbf{K}^\top}{\sqrt{d}}\right),$$

¹ Vaswani A. et al. *Attention Is All You Need*. NeurIPS, 2017.

где softmax применяется по строкам матрицы скоров. Вычислительный граф:

$$\mathbf{S} = \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} \rightarrow \mathbf{P} = \text{softmax}(\mathbf{S}) \rightarrow \mathbf{O} = \mathbf{P}\mathbf{V}.$$

На обучении к $\mathbf{O}$ приходит $\bar{\mathbf{O}} = \partial L / \partial \mathbf{O}$ от следующего слоя (или от лосса). Задача AD — получить $\bar{\mathbf{Q}}, \bar{\mathbf{K}}, \bar{\mathbf{V}}$ (и передать их к предшествующим линейным слоям).



В этом примере есть матричные умножения, нелинейность по строкам и промежуточный тензор $\mathbf{P} \in \mathbb{R}^{n \times n}$, который при больших n доминирует в памяти при использовании reverse-подхода. Именно для решения этой проблемы были предложены методы FlashAttention^a и selective checkpointing^b.

^aDao, T. Fu D., Ermon S., Rudra A., Ré C. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS, 2022. [arXiv:2205.14135](https://arxiv.org/abs/2205.14135); см. также Dao T. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. ICLR, 2024. [arXiv:2307.08691](https://arxiv.org/abs/2307.08691).

^bKorthikanti V. A. et al. *Reducing Activation Recomputation in Large Transformer Models* (selective activation recomputation / SAC). MLSys, 2023. [arXiv:2205.05198](https://arxiv.org/abs/2205.05198).

Обратный режим (VJP через attention)

Прямой проход сохраняет то, что нужно локальным VJP, — как минимум $\mathbf{P}$ (и часто $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ или эквивалент). Старт: дан $\bar{\mathbf{O}} \in \mathbb{R}^{n \times d}$. Идём назад по рёбрам.

Рабочий язык тот же, что у матричных дифференциалов: линейное приращение dL записывают как скалярное произведение

$$\langle \mathbf{A}, \mathbf{B} \rangle = \text{tr}(\mathbf{A}^\top \mathbf{B}) = \sum_{ij} A_{ij} B_{ij},$$

и коэффициент при $d\mathbf{Z}$ будет равен $\bar{\mathbf{Z}}$. На каждом ребре достаточно разложить $dL = \langle \bar{\mathbf{O}}, d\mathbf{O} \rangle$ через $d\mathbf{P}, d\mathbf{V}, \dots$ и посчитать коэффициенты.

1. Ребро $\mathbf{O} = \mathbf{P}\mathbf{V}$. Дифференциал произведения

$$d\mathbf{O} = (d\mathbf{P})\mathbf{V} + \mathbf{P}(d\mathbf{V}).$$

Подставляем в пару с уже известным $\bar{\mathbf{O}}$:

$$\langle \bar{\mathbf{O}}, d\mathbf{O} \rangle = \langle \bar{\mathbf{O}}, (d\mathbf{P})\mathbf{V} \rangle + \langle \bar{\mathbf{O}}, \mathbf{P}(d\mathbf{V}) \rangle.$$

Циклическое свойство следа (или проверка по координатам) даёт два тождества

$$\langle \bar{\mathbf{O}}, (d\mathbf{P})\mathbf{V} \rangle = \langle \bar{\mathbf{O}}\mathbf{V}^\top, d\mathbf{P} \rangle, \quad \langle \bar{\mathbf{O}}, \mathbf{P}(d\mathbf{V}) \rangle = \langle \mathbf{P}^\top \bar{\mathbf{O}}, d\mathbf{V} \rangle.$$

Значит,

$$\langle \bar{\mathbf{O}}, d\mathbf{O} \rangle = \langle \bar{\mathbf{O}}\mathbf{V}^\top, d\mathbf{P} \rangle + \langle \mathbf{P}^\top \bar{\mathbf{O}}, d\mathbf{V} \rangle,$$

и по определению

$$\bar{\mathbf{P}} = \bar{\mathbf{O}}\mathbf{V}^\top, \quad \bar{\mathbf{V}} = \mathbf{P}^\top \bar{\mathbf{O}}.$$

ЗАМЕЧАНИЕ

Интуиция: $\mathbf{O} = \mathbf{P}\mathbf{V}$ — умножение «слева на $\mathbf{P}$, справа на $\mathbf{V}$ ». Градиент по левому множителю получается умножением $\bar{\mathbf{O}}$ справа на $\mathbf{V}^\top$; градиент по правому —

умножением $\bar{\mathbf{O}}$ слева на $\mathbf{P}^\top$. Это ровно VJP для матричного умножения: одна и та же формула, что в autograd для $\mathbf{P} @ \mathbf{V}$.

2. Ребро $\mathbf{P} = \text{softmax}(\mathbf{S})$ по строкам. Softmax действует независимо на каждую строку, поэтому достаточно вывести результат для одной строки $\mathbf{p} = \text{softmax}(\mathbf{s})$ с $\bar{\mathbf{p}}$ (строка матрицы $\bar{\mathbf{P}}$). Дифференциал softmax:

$$d\mathbf{p} = (\text{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^\top) d\mathbf{s} = \mathbf{p} \odot (d\mathbf{s} - \langle \mathbf{p}, d\mathbf{s} \rangle \mathbf{1}),$$

где $\odot$ — поэлементное умножение. Тогда

$$\langle \bar{\mathbf{p}}, d\mathbf{p} \rangle = \langle (\text{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^\top) \bar{\mathbf{p}}, d\mathbf{s} \rangle,$$

потому что матрица $\text{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^\top$ симметрична. Отсюда

$$\bar{\mathbf{s}} = \mathbf{p} \odot (\bar{\mathbf{p}} - \langle \mathbf{p}, \bar{\mathbf{p}} \rangle \mathbf{1}) = (\text{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^\top) \bar{\mathbf{p}}.$$

Построчно собираем $\bar{\mathbf{S}} \in \mathbb{R}^{n \times n}$. Якобиан softmax по строке имеет ранг не выше $n - 1$ и применяется к вектору за $\Theta(n)$ **без** сборки матрицы $n \times n$.

3. Ребро $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top / \sqrt{d}$. Снова дифференциал произведения (с константой $1/\sqrt{d}$):

$$d\mathbf{S} = \frac{(d\mathbf{Q})\mathbf{K}^\top + \mathbf{Q}(d\mathbf{K})^\top}{\sqrt{d}}.$$

Пара с $\bar{\mathbf{S}}$:

$$\langle \bar{\mathbf{S}}, d\mathbf{S} \rangle = \left\langle \frac{\bar{\mathbf{S}}\mathbf{K}}{\sqrt{d}}, d\mathbf{Q} \right\rangle + \left\langle \frac{\bar{\mathbf{S}}^\top \mathbf{Q}}{\sqrt{d}}, d\mathbf{K} \right\rangle,$$

где второе слагаемое получается из $\langle \bar{\mathbf{S}}, \mathbf{Q}(d\mathbf{K})^\top \rangle = \langle \bar{\mathbf{S}}^\top \mathbf{Q}, d\mathbf{K} \rangle$. Следовательно,

$$\bar{\mathbf{Q}} = \frac{\bar{\mathbf{S}}\mathbf{K}}{\sqrt{d}}, \quad \bar{\mathbf{K}} = \frac{\bar{\mathbf{S}}^\top \mathbf{Q}}{\sqrt{d}}.$$



Общий шаблон на каждом ребре один и тот же: (i) выписать дифференциал от выхода $d(\text{выход})$ через дифференциалы входов; (ii) подставить в формулу $\langle \text{выход}, d(\text{выход}) \rangle$; (iii) перекинуть множители в скалярном произведении и получить вход. Так устроен любой матричный VJP в AD; attention — просто композиция трёх таких рёбер.

Один обратный проход даёт сразу все три величины $\bar{\mathbf{Q}}, \bar{\mathbf{K}}, \bar{\mathbf{V}}$. Если $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$ и так далее, ещё один VJP через линейные слои даёт градиенты по $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ и по входу $\mathbf{X}$. Это по-прежнему один reverse-проход относительно скалярного лосса.

ЗАМЕЧАНИЕ

Величина $\bar{v}$ — это $\partial L / \partial v$ после уже посчитанного прямого прохода. В PyTorch тот же объект называют градиентом тензора; `loss.backward()` расставляет градиенты от скалярного лосса к листьям с флагом `requires_grad=True`. Для механизма внимания фреймворк делает цепочку преобразований, описанную выше (или её fused-вариант в ядре)¹.

ПРИМЕР 2

Сложность прямого прохода в механизме внимания $T = \text{OPS}\{\mathbf{O}\} = \Theta(n^2 d)$ (доминанта — $\mathbf{QK}^\top$ и $\mathbf{PV}$). Reverse-подход для $\bar{\mathbf{Q}}, \bar{\mathbf{K}}, \bar{\mathbf{V}}$ имеет ту же асимптотику $\Theta(n^2 d)$. Память наивного reverse — $\Theta(n^2)$ на матрицу $\mathbf{P}$ (или $\mathbf{S}$), плюс активации остальных слоёв. При $n \sim 10^4$ – 10^5 именно эта матрица $n \times n$ становится узким местом: отсюда rematerialization, неполная материализация внимания (FlashAttention) и selective checkpointing внутри блока.

Softmax-VJP использует структуру $\text{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^\top$ и применяется к $\bar{\mathbf{p}}$ за линейное время по длине строки — без явной сборки якобиана softmax.

4. СЛОЖНОСТЬ РЕЖИМОВ И ВЫБОР

Пусть $L: \mathbb{R}^n \rightarrow \mathbb{R}$ стоит $T = \text{OPS}\{L\}$, а для отображения $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ нужен полный якобиан $\mathbf{J} \in \mathbb{R}^{m \times n}$.

	Forward	Reverse
Один проход даёт	столбец $\mathbf{J}$ (JVP)	строку $\mathbf{J}$ (VJP)
Полный $\mathbf{J}$	$\mathcal{O}(nT)$	$\mathcal{O}(mT)$
Память	как у f	промежуточные активации
Типичный случай	$n \ll m$	$m \ll n$

Более аккуратно (Griewank & Walther, *Evaluating Derivatives*, SIAM, 2008): один reverse для скалярного выхода удовлетворяет $\text{OPS}\{L'\} \leq \omega T$ с $\omega \leq 5$; один JVP — аналогичная константа ω_f . Память reverse — $\Theta(\#\{v_i\})$ сохранённых активаций (плюс структура графа).



Практическое правило: смотри на размеры «дифференцируемых входов» и «выходов отображения». Обучение сети: $n \sim 10^6$ – 10^{12} , $m = 1$ (скалярный лосс) — только reverse. Полный якобиан векторного выхода по малому числу параметров/гиперпараметров — forward (`jax.jacfwd`, `torch.func.jacfwd`).

ПРИМЕР 3

В экосистеме JAX явно разделены примитивы: `jax.grad` / `jax.vjp` — reverse, `jax.jvp` — forward, `jacfwd/jacrev` — полный якобиан выбранным режимом. В PyTorch классический путь — reverse через `backward`; forward и функциональные якобианы — через `torch.autograd.functional` и `torch.func`.

¹ Fused (слитый) kernel / operator fusion: несколько логических шагов графа (например, $\mathbf{QK}^\top$, масштабирование, softmax, умножение на $\bar{\mathbf{V}}$ и куски их VJP) выполняют внутри одного GPU-ядра, не записывая полные промежуточные тензоры. Это экономит память и пропускную способность памяти ценой более сложного ядра; см. Chen T. et al. *TVM: An Automated End-to-End Optimizing Compiler for Deep Learning*. OSDI, 2018 ([arXiv:1802.04799](https://arxiv.org/abs/1802.04799)), раздел про operator fusion; для внимания — FlashAttention (Dao et al., 2022; см. ссылку выше).

Гессиан-вектор без хранения гессиана

Гессиан $f''(\mathbf{x}) \in \mathbb{R}^{n \times n}$ при большом n не хранят. Нужно произведение

$$f''(\mathbf{x}) \mathbf{v} = \nabla_{\mathbf{x}} \langle f'(\mathbf{x}), \mathbf{v} \rangle$$

считается композицией: сначала градиент (reverse или через JVP), затем ещё один reverse по скаляру $g(\mathbf{x}) = \langle f'(\mathbf{x}), \mathbf{v} \rangle$. Стоимость — порядка двух градиентов, а не $\mathcal{O}(n^2)$. Это стандартный оракул для Ньютона-CG и связанных методов¹.

Если якобиан берётся не по входу $\mathbf{x}$, а по параметрам сети $\boldsymbol{\theta} \in \mathbb{R}^P$ при фиксированном $\mathbf{x}$, роль n играет именно P : полный $\mathbf{J} \in \mathbb{R}^{m \times P}$ стоит $\mathcal{O}(PT)$ в forward и $\mathcal{O}(mT)$ в reverse. У MLP с плотным последним слоем P растёт с m (веса слоя $\text{Linear}(h, m)$ уже дают $\sim hm$ параметров), поэтому часто $P \gg m$ и reverse остаётся предпочтительным.

5. ЕСЛИ КРАТКО (AD)

- Конечные разности: $\mathcal{O}(dT)$ и чувствительность к ε ; символьное дифференцирование плохо масштабируется на программы.
- AD считает производные по вычислительному графу: forward — столбцы через JVP, reverse — строки через VJP.
- Для скалярного лосса reverse даёт полный градиент за $\mathcal{O}(T)$, forward — за $\mathcal{O}(dT)$.
- Режим для полного якобиана выбирают по $\min(n, m)$; обучение моделей — reverse.
- $\mathbf{H}\mathbf{v}$ получают двумя проходами AD без сборки гессиана.

Что почитать

- Baydin A. G., Pearlmutter B. A., Radul A. A., Siskind J. M. *Automatic differentiation in machine learning: a survey*. JMLR, 18(153):1–43, 2018 (jmlr.org). Сравнение численных разностей, символьного дифференцирования и AD.
- Moses J. *Algebraic simplification: a guide for the perplexed*. CACM, 14(8):527–537, 1971. Промежуточное раздувание выражений в CAS.
- Geddes K. O., Czapor S. R., Labahn G. *Algorithms for Computer Algebra*. Springer, 1992. Алгоритмические основы символьных вычислений.
- Meurer A. et al. *SymPy: symbolic computing in Python*. PeerJ Computer Science, 3:e103, 2017 ([doi:10.7717/peerj-cs.103](https://doi.org/10.7717/peerj-cs.103)).
- Griewank A., Walther A. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. SIAM, 2008.
- Dao T., Fu D., Ermon S., Rudra A., Ré C. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS, 2022 ([arXiv:2205.14135](https://arxiv.org/abs/2205.14135)).
- Dao T. *FlashAttention-2*. ICLR, 2024 ([arXiv:2307.08691](https://arxiv.org/abs/2307.08691)).
- Korthikanti V. A. et al. *Reducing Activation Recomputation in Large Transformer Models*. MLSys, 2023 ([arXiv:2205.05198](https://arxiv.org/abs/2205.05198)). Selective activation checkpointing / recomputation.

¹ Произведение гессиана на вектор через AD без сборки $\mathbf{H}$: Pearlmutter B. A. *Fast Exact Multiplication by the Hessian*. Neural Computation, 6(1):147–160, 1994. Newton-CG / Hessian-free обучение нейросетей: Martens J. *Deep learning via Hessian-free optimization*. ICML, 2010 ([pdf](#)); Martens J., Sutskever I. *Learning Recurrent Neural Networks with Hessian-Free Optimization*. ICML, 2011 ([pdf](#)).

- Chen T. et al. *TVM: An Automated End-to-End Optimizing Compiler for Deep Learning*. OSDI, 2018 ([arXiv:1802.04799](https://arxiv.org/abs/1802.04799)). Operator / kernel fusion в компиляторах DL.
- Pearlmutter B. A. *Fast Exact Multiplication by the Hessian*. Neural Computation, 6(1):147–160, 1994. Hessian-vector product через AD.
- Martens J. *Deep learning via Hessian-free optimization*. ICML, 2010 ([pdf](#)). Newton-CG для обучения нейросетей.
- Martens J., Sutskever I. *Learning Recurrent Neural Networks with Hessian-Free Optimization*. ICML, 2011 ([pdf](#)).
- HIPS Autograd tutorial: github.com/HIPS/autograd.
- Мини-реализация AD: github.com/mattjj/autodidact.
- PyTorch checkpoint: pytorch.org/docs/stable/checkpoint.html.
- JAX remat: docs.jax.dev — [Autodiff cookbook](#).